| USN | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|

"24CS330"

**JSS MAHAVIDYAPEETHA**
**JSS SCIENCE AND TECHNOLOGY UNIVERSITY, MYSURU**

*Department of Computer Science and Engineering*

**III Semester: Test II**

**Data Structures Model Answers**

| Date: 08-01-2026 | Max. Marks: 30 Marks | **Q1 is compulsory. Choice: Q2 or Q3** |
|---|---|---|

1. a. Develop a C function to reverse the contents of single linked list of integer numbers. **(5m)**

```
Node* reverseList(struct Node* head)
{
  if (head == NULL)
  {
    printf("List is empty. Cannot reverse.\n");
    return head;
  }
  struct Node *FP = head, *LN = NULL, *temp;
  while (FP != NULL)
  {
    temp= FP;
    FP=FP->next;
    temp->next=LN;
    LN=temp;
  }
  printf("List has been reversed successfully.\n");
  return LN;
}
```

1. b. Illustration with an example, memory representation of circular single linked list and double linked list. **(2.5 X 2 = 5m)**

**Double linked list:**

Head → 1

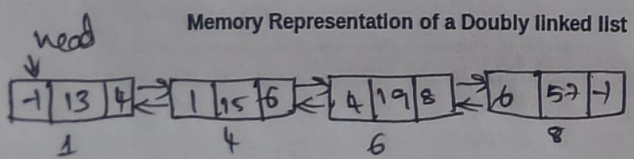
| | Data | Prev | Next |
|---|---|---|---|
| 1 | 13 | -1 | 4 |
| 2 | | | |
| 3 | | | |
| 4 | 15 | 1 | 6 |
| 5 | | | |
| 6 | 19 | 4 | 8 |
| 7 | | | |
| 8 | 57 | 6 | -1 |

**Circular Single Linked List:**

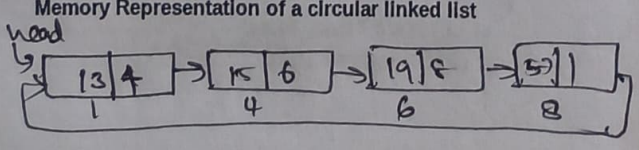
start → 1

| | Data | Next |
|---|---|---|
| 1 | 13 | 4 |
| 2 | | |
| 3 | | |
| 4 | 15 | 6 |
| 5 | | |
| 6 | 19 | 8 |
| 7 | | |
| 8 | 57 | 1 |

**Memory Representation of a Doubly linked list**



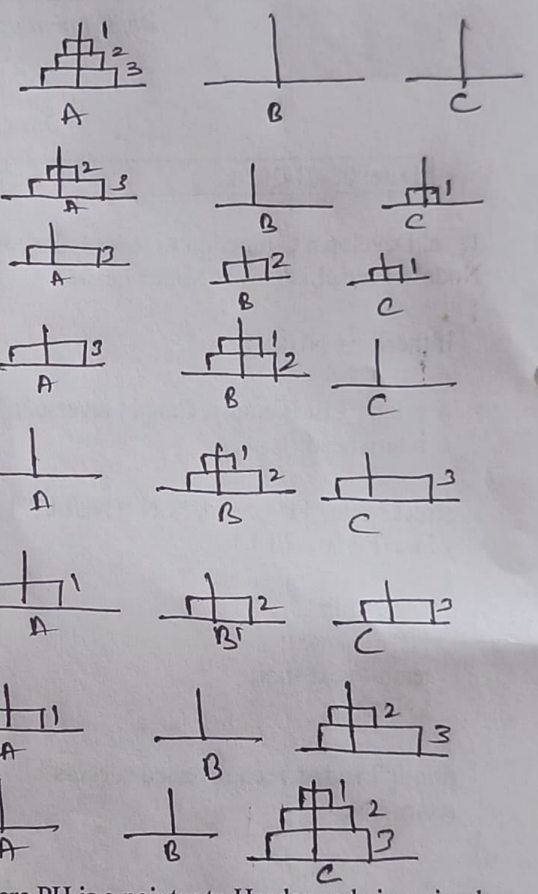

**Memory Representation of a circular linked list**



1. c. Define recursion. Develop a recursive function for tower of Hanoi and trace the same for 3 disks. **(5m)**

Recursion is a programming technique in which a function calls itself to solve a problem by breaking it down into smaller sub-problems, until a base condition is reached that stops further calls.

```
void tower(int n, char source, char dest, char aux)
{
  if (n == 1)
  {
    printf("Move disk 1 from %c to %c\n", source, dest);
    return;
  }
  tower(n - 1, source, aux, dest);
  printf("Move disk %d from %c to %c\n", n, source, dest);
  tower(n - 1, aux, dest, source);
}
```




**Trace for 3 Disks**

**Call:** tower(3, A, C, B)

**Step-by-Step Moves**

| Step | Move |
|---|---|
| 1 | Move disk 1 from A → C |
| 2 | Move disk 2 from A → B |
| 3 | Move disk 1 from C → B |
| 4 | Move disk 3 from A → C |
| 5 | Move disk 1 from B → A |
| 6 | Move disk 2 from B → C |
| 7 | Move disk 1 from A → C |

2. a. Write a function called DELETE_APPEND (PH), where PH is a pointer to Header node in a circular single linked list of integers, which deletes the front node and appends it to the end of the list. Before writing the function, define node structure, create and initialize an empty list.

**Structure Definition: (1m)**

```
struct node
{
  int data;
  struct node *next;
};
```

**Creating and Initializing an Empty Circular Singly Linked List: (1m)**

```
struct node *PH;
PH = (struct node *)malloc(sizeof(struct node));
PH->next = PH;
```

```
void DELETE_APPEND(struct node *PH)  (3m)
{
  struct node *first, *last;
  if (PH->next == PH)
  {
```

```
    printf("List is empty. Operation not possible.\n");
    return;
  }
  if (PH->next->next == PH)
  {
    printf("Only one node present. No change made.\n");
    return;
  }
  first = PH->next;        // Front node
  last = PH;     // Find last node
  while (last->next != PH)
    last = last->next;
  PH->next = first->next;    // Delete front node
  last->next = first;          // Append deleted node at the end
  first->next = PH;
}
```

2. b. Construct the algorithm to convert a given infix expression to postfix form and trace the stacks content for the expression $A+(B*(C-D)E^F)/G$.

**Algorithm: Infix to Postfix:**

- Initialize an empty stack for operators and an empty list for the output.
- Read the infix expression from left to right.
- If the character is an operand (numbers or variables):
  - Add it directly to the output list.
- If the character is an opening parenthesis (:
  - Push it onto the stack.
- If the character is a closing parenthesis ):
  - Pop from the stack to the output list until an opening parenthesis ( is encountered at the top of the stack. Discard the pair of parentheses.
- If the character is an operator (like +, -, *, /, etc.):
  - While the stack is not empty and the top of the stack has an operator of higher or equal precedence than the current operator, pop operators from the stack to the output list.
  - Push the current operator onto the stack.
- After scanning the infix expression, pop any remaining operators from the stack and append them to the output list.
- The output list now contains the postfix expression.

**Soution: A + (B * (C – D) * E ^ F) / G**

| Symbol | Stack | Postfix |
|---|---|---|
| A | — | A |
| + | + | A |
| ( | +( | A |
| B | +( | AB |
| * | +(* | AB |
| ( | +(*( | AB |
| C | +(*( | ABC |
| – | +(*(– | ABC |
| D | +(*(– | ABCD |
| ) | +(* | ABCD– |
| * | +(* | ABCD–* |

| | | |
|---|---|---|
| E | +( * | ABCD−*E |
| ^ | +( * ^ | ABCD−*E |
| F | +( * ^ | ABCD−*EF |
| ) | + | ABCD−*EF^* |
| / | +/ | ABCD−*EF^* |
| G | +/ | ABCD−*EF^*G |
| End | — | ABCD−*EF^*G/+ |

Answer: ABCD−*E+F^*G/+

3. a. Write a function called SEARCH-INSERT(PH, X) where PH is a pointer to Header node in a circular double linked list of integers and X is an integer value which searches for X and if it does not exist, appends it to the end of list.

**Structure Definition:**

```
struct node
{
   int data;
   struct node *prev;
   struct node *next;
};
```

**Creating and Initializing an Empty List:**

```
struct node *PH;
PH = (struct node *)malloc(sizeof(struct node));
PH->next = PH;
PH->prev = PH;
```

```
void SEARCH_INSERT(struct node *PH, int X)
{
   struct node *temp, *newNode;

   // Traverse the list to search for X
   temp = PH->next;
   while (temp != PH)
   {
      if (temp->data == X)
      {
         printf("Element %d found in the list.\n", X);
         return;
      }
      temp = temp->next;
   }

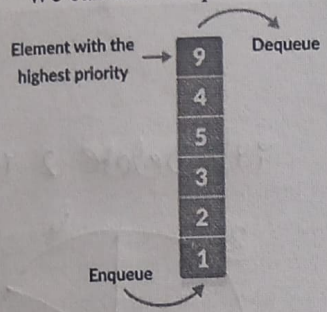
   // X not found → insert at end
   newNode = (struct node *)malloc(sizeof(struct node));
   newNode->data = X;

   // Insert before header (at the end)
   newNode->next = PH;
   newNode->prev = PH->prev;
   PH->prev->next = newNode;
   PH->prev = newNode;

   printf("Element %d not found. Inserted at the end.\n", X);
}
```

3. b. Discuss priority queue.

- A priority queue is a **special type of queue** in which each element is associated with a **priority value**. And, elements are served on the basis of their priority. That is, higher priority elements are served first.
- However, if elements with the same priority occur, they are served according to their order in the queue.
- **Assigning Priority Value**
- Generally, the value of the element itself is considered for assigning the priority. For example, the element with the highest value is considered the highest priority element. However, in other cases, we can assume the element with the lowest value as the highest priority element.
- We can also set priorities according to our needs.



Removing Highest Priority Element

**Implementation of Priority Queue:**

- Priority queue can be implemented using an array, a linked list, a heap data structure, or a binary search tree.
- Among these data structures, heap data structure provides an efficient implementation of priority queues.
- Method 1: Insert in any order and delete min/max element. This requires shifting after deletion in array representation to fill up the space created.
- Method 2: Insert By order and delete front most item as the highest prioritized item will be at the front of the queue. This method requires shifting during insertion.
- Both of the above discussed methods are not efficient as shifting is costlier in terms of time.
- Note: More efficient method of implementing priority queue is using Heap data structure.
- A comparative analysis of different implementations of priority queue is given below.

| Operations | peek | insert | delete |
|---|---|---|---|
| Linked List | O(1) | O(n) | O(1) |
| Binary Heap | O(1) | O(log n) | O(log n) |
| Binary Search Tree | O(1) | O(log n) | O(log n) |

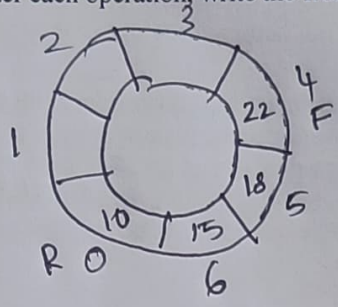
**Applications of Priority Queue:**

- CPU Scheduling
- Graph Algorithms – Prim's, Dijkstra's
- Data Compression (Huffman Coding)
- Event-Driven Simulation
- Network Traffic Management
- AI and Pathfinding (Algorithms like A* use priority queues).

3. c. A circular queue of size 7 contains four elements 22, 18, 15, 10 (inserted in the same sequence). The front index F = 4. Starting with this status, show the queue contents using circular representation while performing the following operations. After each operation, write the front item and rear item.

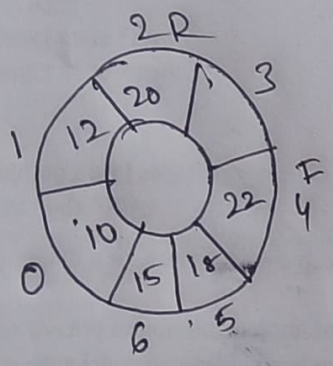
Operations:

i. Insert 12, 20
ii. Delete 2 items
iii. Insert 25, 30, 35
iv. Delete 1 item
v. Insert 40



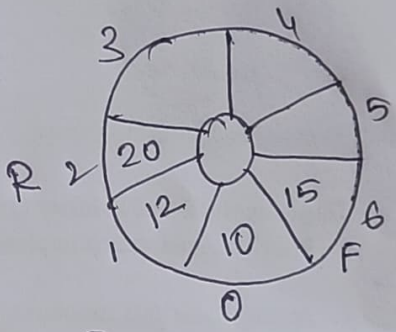

Items
F → 22
R → 10

i) Insert 12, 20



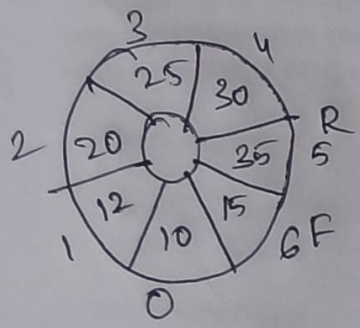

F → 22
R → 20

ii) Delete 2 items



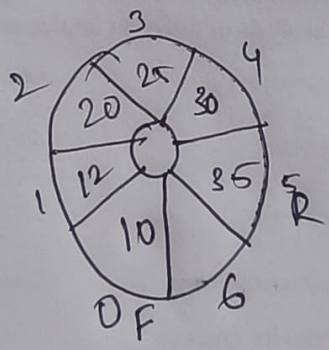

F = 15
R = 20

iii) Insert 25, 30, 35



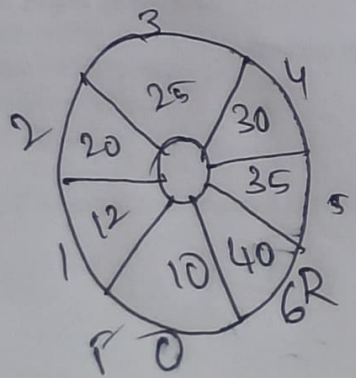

F = 15
R = 35

iv) Delete 1 item




F = 10
R = 35

v) Insert 40




F = 10
R = 40